Institute of Geographical Information Systems

CS-250 - Data Structures and Algorithms LAB

Semester: Spring 2026	**Class: SCEE-IGIS - 2024**
Name: Ali Nawaz	**CMS ID : 00000526123**
Submitted to: Ma'am Alvina Anjum	**Due Date: April 15, 2026**

LAB 10: Graphs

Part A: Graph Representation (Adjacency Matrix)

Concept

- A graph with V vertices is represented using a V × V matrix.
- adj[i][j] = 1 if there is an edge from vertex i to j, otherwise 0.

Task 1: Create an Adjacency Matrix

Write a program to:

- Take number of vertices V as input.
- Allow the user to insert and delete edges.
- Display the updated adjacency matrix.

Algorithm: Insertion of Edge

```
InsertEdge(u, v):
   adj[u][v] = 1
   adj[v][u] = 1  // For undirected graphs
```

Algorithm: Deletion of Edge

```
DeleteEdge(u, v):
   adj[u][v] = 0
   adj[v][u] = 0  // For undirected graphs
```

C++ Code: Graph with Insertion & Deletion

Write the code for inserting and deleting a node in a graph. Some hints are provided to you to help you write the code.

```
const int MAX = 100;
int adj[MAX][MAX];  // Adjacency matrix

void insertEdge(int u, int v) {

}
// to delete an edge
void deleteEdge(int u, int v) {

}


void displayMatrix(int V) {

}
```

- Define a Node class with data, left, and right child pointers.
- Write a function to create a binary tree from user input (recursive or static).

Part B: BFS Traversal Using Adjacency Matrix

Algorithm: BFS (Adjacency Matrix)

```
BFS(start):
   Create visited[V] array and initialize all to false
   Create empty queue
   Mark visited[start] = true, enqueue start

   While queue is not empty:
      current = dequeue
      print current
      For all vertices i:
         if adj[current][i] == 1 and visited[i] == false:
            visited[i] = true
            enqueue i
```

C++ Code: BFS Using Adjacency Matrix
Write the code to traverse the graph using breadth-first search algorithm using the hints provided below:

```
//adjacency matrix and visited array
int adj[MAX][MAX];
bool visited[MAX];

//function to carry out BFS

void BFS(int start, int V) {

// loop till queue is empty
   while (!q.empty()) {

//loop to all vertices that have not been visited till all vertices have been visited and remove from

      for (int i = 0; i < V; ++i) {
         if (adj[curr][i] == 1 && !visited[i]) {
            visited[i] = true;
```

Sample Lab Flow

Enter number of vertices: 5

Insert edge: 0 1
Insert edge: 0 2
Insert edge: 1 3
Insert edge: 2 4

Display Matrix

Delete edge: 0 2

BFS from node 0

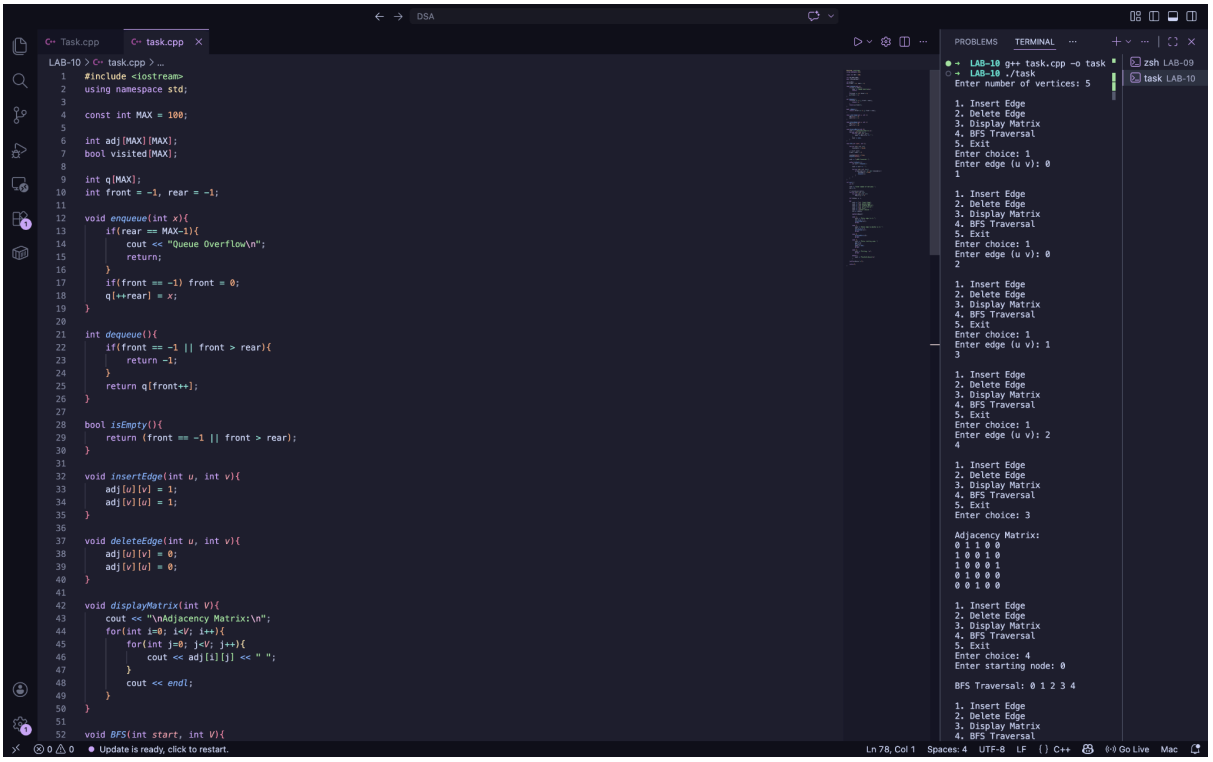
Student Deliverables

- Source code for insertion, deletion, and BFS.
- Screenshot of adjacency matrix before and after deletion.
- Output of BFS traversal.
- "What happens to BFS traversal if a node is disconnected?"

Screenshot of adjacency matrix before and after deletion:

```
1. Insert Edge
2. Delete Edge
3. Display Matrix
4. BFS Traversal
5. Exit
Enter choice: 3

Adjacency Matrix:
0 1 1 0 0
1 0 0 1 0
1 0 0 0 1
0 1 0 0 0
0 0 1 0 0

1. Insert Edge
2. Delete Edge
3. Display Matrix
4. BFS Traversal
5. Exit
Enter choice: 2
Enter edge to delete (u v): 0
2

1. Insert Edge
2. Delete Edge
3. Display Matrix
4. BFS Traversal
5. Exit
Enter choice: 3

Adjacency Matrix:
0 1 0 0 0
1 0 0 1 0
0 0 0 0 1
0 1 0 0 0
0 0 1 0 0
```

Output of BFS traversal:



Q: What happens to BFS traversal if a node is disconnected?
Ans: If a node is disconnected, BFS will not visit it.
Because:

- BFS only goes through connected nodes
- It cannot reach nodes that have no path from the starting node